

week-1

April 23, 2026

0.1 Import the Packages

```
[1]: from scipy.integrate import quad
import numpy as np
import matplotlib.pyplot as plt
```

0.2 Define the parameter Space

0.2.1 Set the range of z

$z \in (0.001, 2)$

```
[2]: z_list = np.linspace(0.001,2,10000)
```

0.2.2 Set the fixed values of the parameter (measured values)

$\Omega_m = 0.3$, $H_0 = 67$, $c = 1$

```
[3]: O_m_fixed = 0.3
H0_fixed = 67
c = 1
```

0.2.3 Specify the range of variation of Ω_m and H_0

$\Omega_m \in (0.1, 0.9)$, $H_0 \in (60, 80)$

```
[4]: O_m_list = np.linspace(0.1,0.9,5)
H0_list = np.linspace(60,80,5)
```

0.3 The dimensionless Hubble function : $E(Z)$

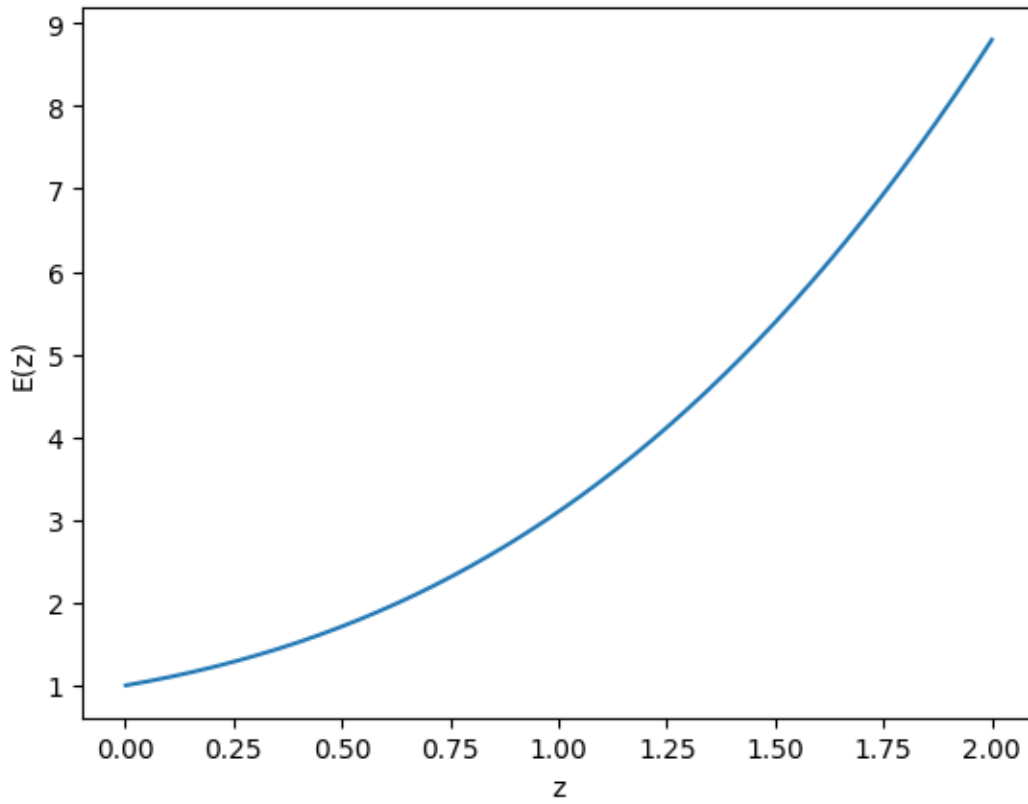
0.3.1 Define the function $E(z)$

$$E(z) = \sqrt{\Omega_m(1+z)^3 + (1 - \Omega_m)}$$

```
[5]: def E(z):
    result = (O_m * ((1+z)**3)) + (1 - O_m)
    return result
```

0.3.2 Plot $E(z)$ vs. z with $\Omega_m = 0.3$

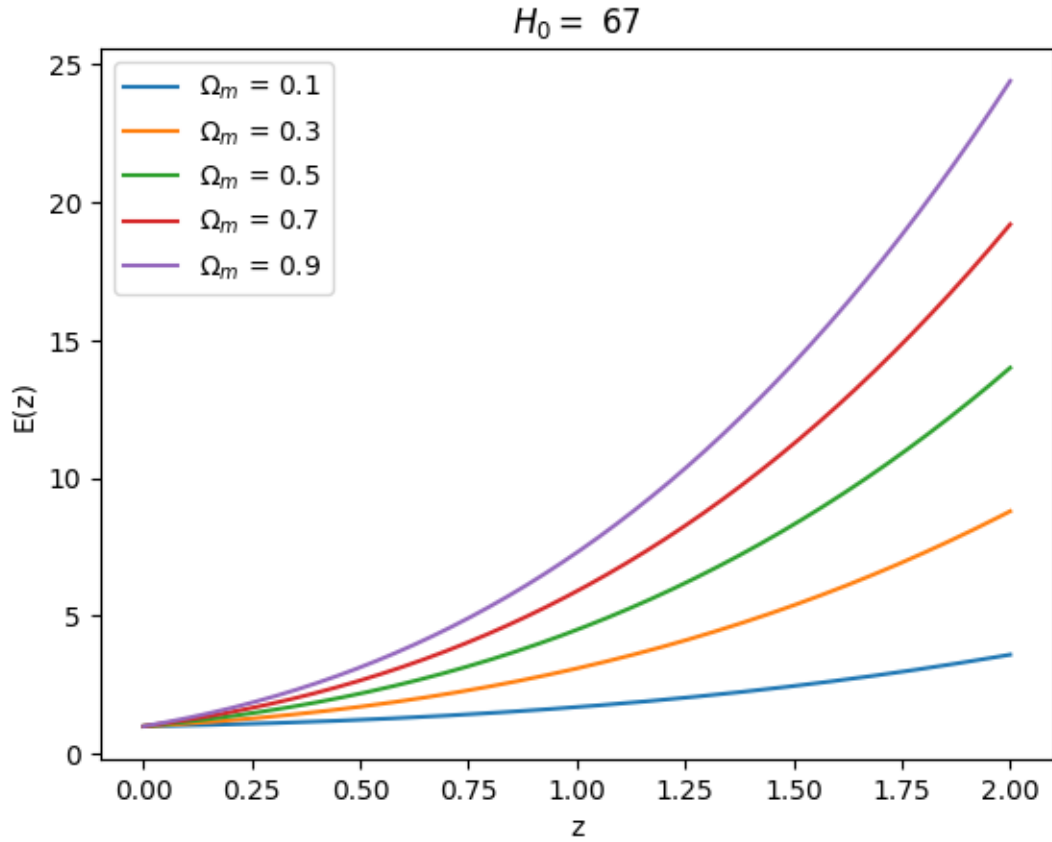
```
[6]: O_m = O_m_fixed
E_list = E(z_list)
plt.xlabel("z")
plt.ylabel("E(z)")
plt.plot(z_list,E_list)
plt.show()
```



0.3.3 Plot $E(z)$ vs. z with $\Omega_m \in (0.1, 0.9)$

```
[7]: for O_m in O_m_list:
    E_diff_O_m_list = []
    H0 = H0_fixed
    for z in z_list:
        E_diff_O_m_list.append(E(z))
    plt.plot(z_list, E_diff_O_m_list, label=rf"$\Omega_m$ = {O_m:.1f}")
plt.legend()
plt.title(fr"$H_0$ = {H0}")
plt.xlabel("z")
plt.ylabel("E(z)")
```

```
plt.show()
```



0.4 The luminosity distance : $d_L(z)$

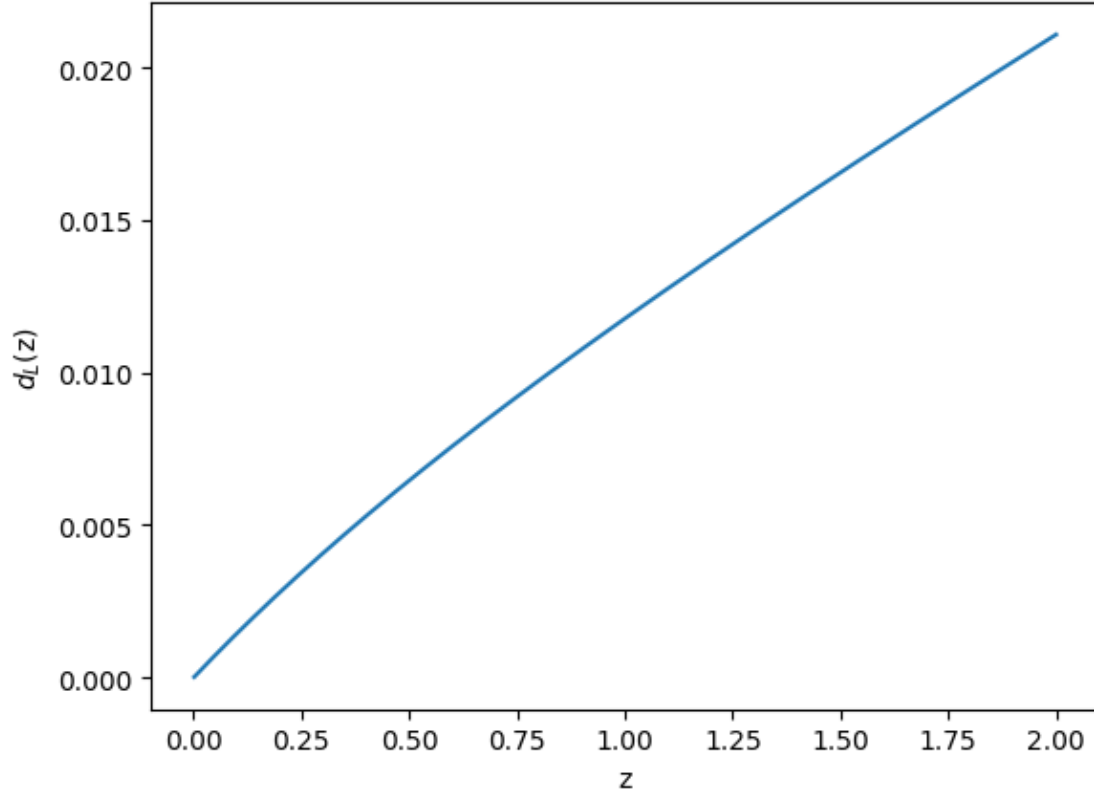
0.4.1 Define the function $d_L(z)$

```
[8]: def d_L(zf):  
    # Increase the limit of subdivisions  
    inte, error = quad(lambda z: 1/E(z), 0, zf, epsabs=1e-5, epsrel=1e-5)  
  
    result = (1+zf) * (c / H0) * inte  
    return result
```

0.4.2 Plot $d_L(z)$ vs. z with $\Omega_m = 0.3$ and $H_0 = 67$

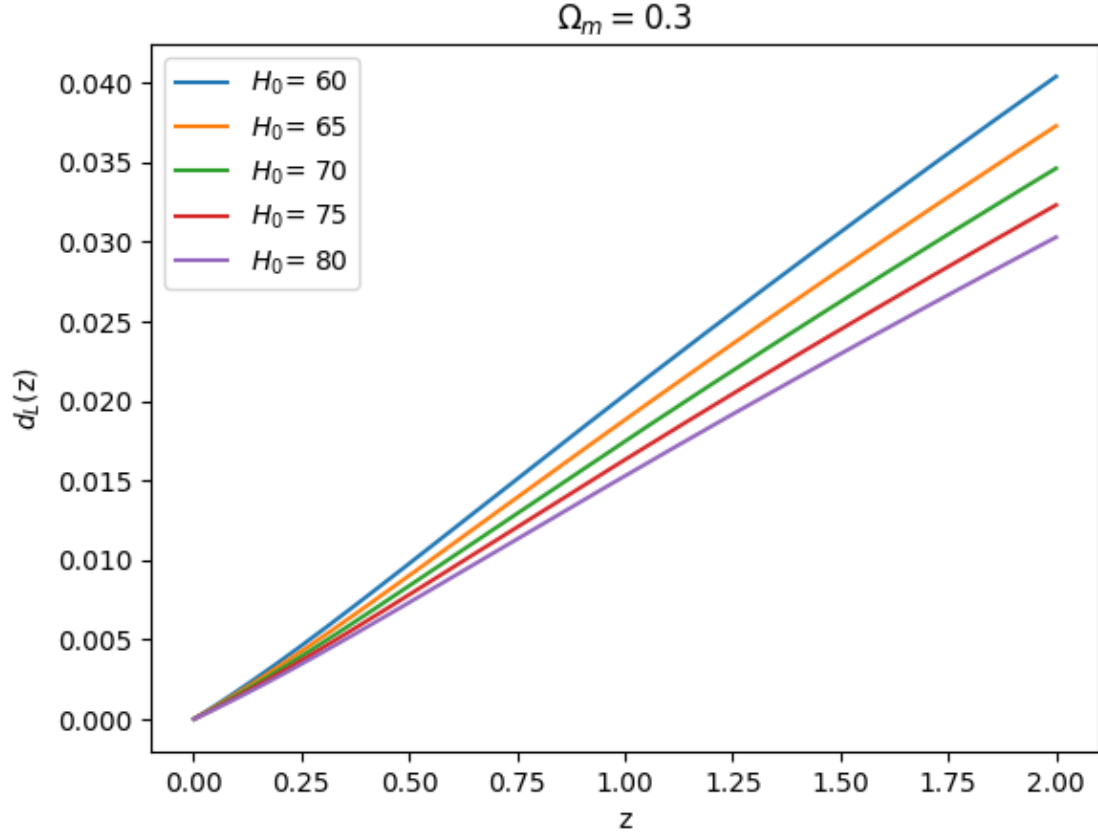
```
[9]: H0 = H0_fixed  
dL_list = []  
for z in z_list:  
    dL_list.append(d_L(z))  
dL_list = np.array(dL_list)
```

```
plt.xlabel("z")
plt.ylabel(r"$d_L(z)$")
plt.plot(z_list,dL_list)
plt.show()
```



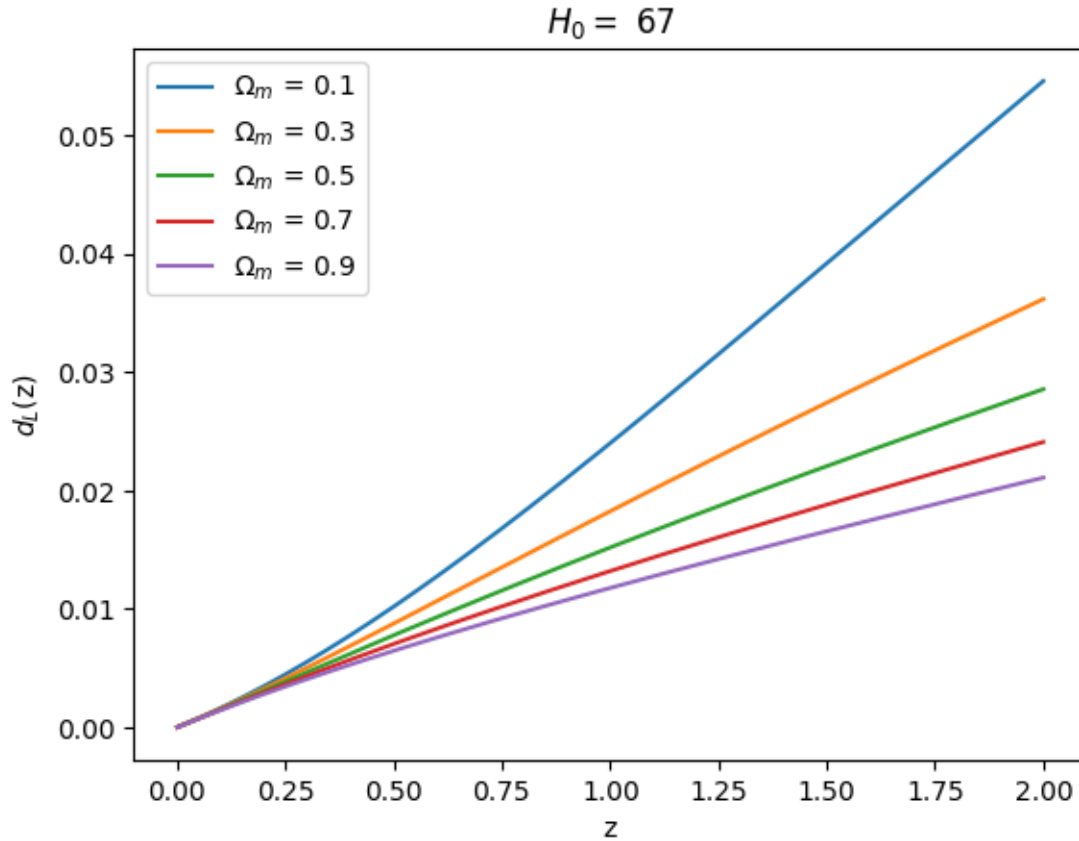
0.4.3 Plot $d_L(z)$ vs. z with $\Omega_m = 0.3$ and $H_0 \in (60, 80)$

```
[10]: for H0 in H0_list:
        dL_diff_H_0_list = []
        O_m = O_m_fixed
        for z in z_list:
            dL_diff_H_0_list.append(d_L(z))
        plt.plot(z_list, dL_diff_H_0_list, label=rf"$H_0$= {H0:.0f}")
    plt.legend()
    plt.title(r"$\Omega_m=0.3$")
    plt.xlabel("z")
    plt.ylabel(r"$d_L(z)$")
    plt.show()
```



0.4.4 Plot $d_L(z)$ vs. z with $\Omega_m \in (0.1, 0.9)$ and $H_0 = 67$

```
[11]: for O_m in O_m_list:
        dL_diff_O_m_list = []
        H0 = H0_fixed
        for z in z_list:
            dL_diff_O_m_list.append(d_L(z))
        plt.plot(z_list, dL_diff_O_m_list, label=rf"\Omega_m$ = {O_m:.1f}")
    plt.legend()
    plt.title(fr"$H_0=$ {H0}")
    plt.xlabel("z")
    plt.ylabel(r"$d_L$(z)")
    plt.show()
```



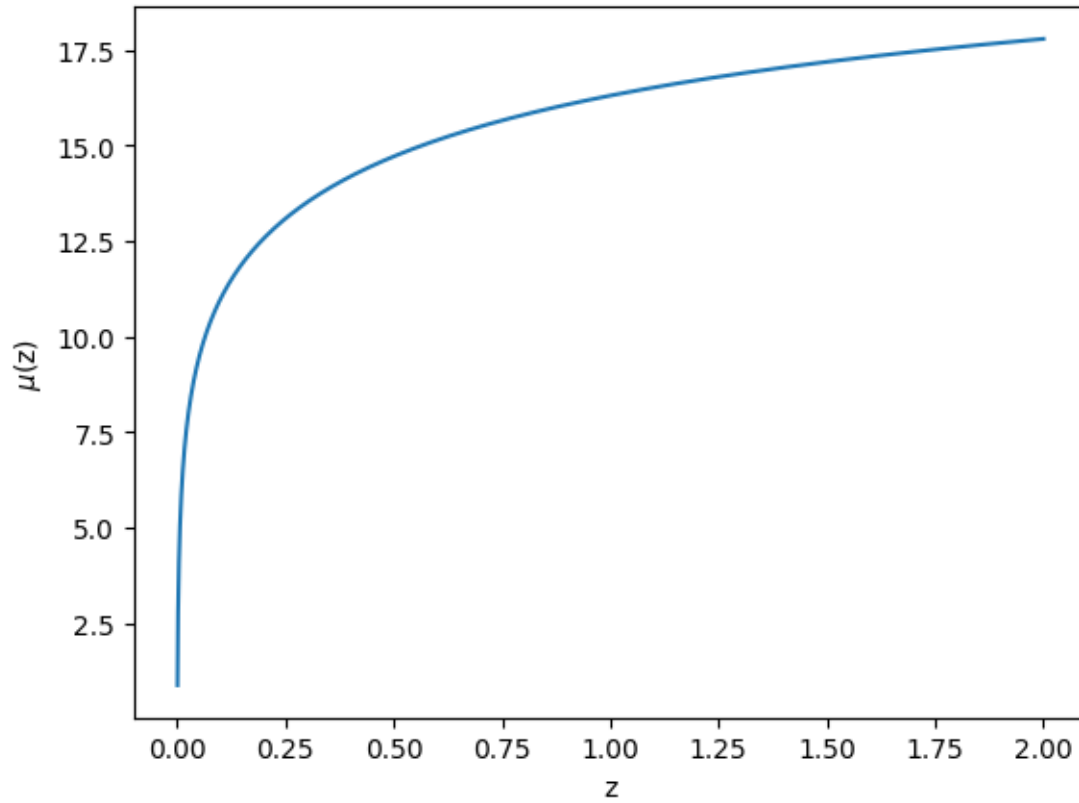
0.5 The theoretical distance modulus : $\mu(z)$

0.5.1 Define the function $\mu(z)$

```
[12]: def mu(z):
        result = (5*np.log10(d_L(z))) + 25
        return result
```

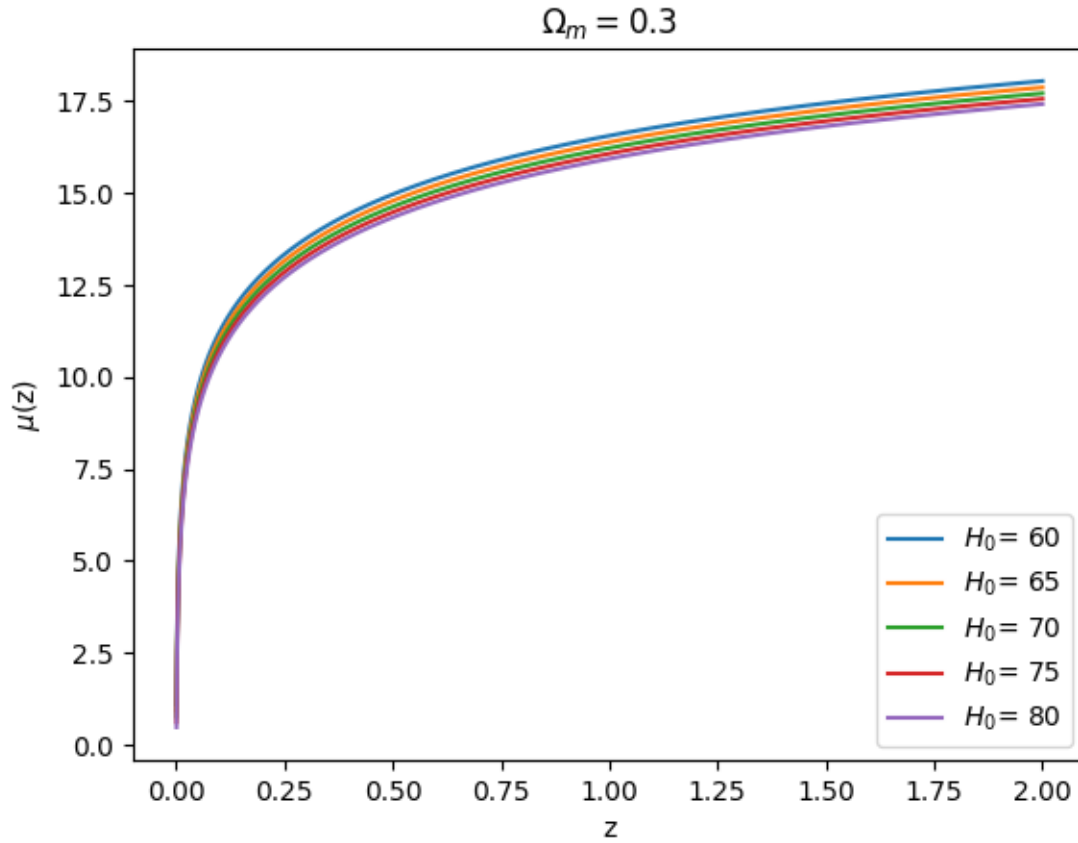
0.5.2 Plot $\mu(z)$ vs. z with $\Omega_m = 0.3$ and $H_0 = 67$

```
[13]: O_m = O_m_fixed
       H0 = H0_fixed
       mu_list = []
       for z in z_list:
           mu_list.append(mu(z))
       mu_list = np.array(mu_list)
       plt.xlabel("z")
       plt.ylabel(r"$\mu$(z)")
       plt.plot(z_list,mu_list)
       plt.show()
```



0.5.3 Plot $\mu(z)$ vs. z with $\Omega_m = 0.3$ and $H_0 \in (60, 80)$

```
[14]: for H0 in H0_list:
        diff_H_0_list = []
        O_m = O_m_fixed
        for z in z_list:
            diff_H_0_list.append(mu(z))
        plt.plot(z_list, diff_H_0_list, label=r"$H_0$= {H0:.0f}")
    plt.legend()
    plt.title(r"$\Omega_m=0.3$")
    plt.xlabel("z")
    plt.ylabel(r"$\mu(z)$")
    plt.show()
```



0.5.4 Plot $\mu(z)$ vs. z with $\Omega_m \in (0.1, 0.9)$ and $H_0 = 67$

```
[15]: for O_m in O_m_list:
        diff_O_m_list = []
        H0 = H0_fixed
        for z in z_list:
            diff_O_m_list.append(mu(z))
        plt.plot(z_list, diff_O_m_list, label=r"$\Omega_m$ = {O_m:.1f}")
plt.legend()
plt.xlabel("z")
plt.ylabel(r"$\mu$(z)")
plt.show()
```